AI 驱动 Web 系统功能/UI 测试最佳实践方法论

> 本文档综合第五届 AI 编程实战大赛全部参赛作品的测试方法论，提炼可复用的最佳实践。
> 适用场景：内网/外网部署的 Web 管理系统（SPA 架构为主），使用 AI 工具 + 浏览器自动化进行功能测试、UI 测试与安全测试。

一、方法论总览：五阶段闭环

```
┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐
│  侦察    │ → │  建模    │ → │  执行    │ → │  研判    │ → │  沉淀    │
│ RECON    │   │ MODEL    │   │ EXECUTE  │   │ TRIAGE   │   │ EVIDENCE │
└──────────┘   └──────────┘   └──────────┘   └──────────┘   └──────────┘
     ↑                                                          │
     └──────────────── 新发现驱动下一轮迭代 ────────────────────────┘
```

核心原则：

1. **先侦察后测试**：不写任何用例之前，先让 AI 用 Playwright 或 curl "看一眼"系统，获取真实 DOM、API 路径和技术栈信息
2. **AI 是大脑，自动化是手脚**：AI 负责测试策略、用例设计、脚本生成、结果分析和报告撰写；Playwright/curl 负责执行和取证
3. **证据链完整**：每个发现都有截图、API 响应、复现命令三重证据
4. **迭代驱动**：浅层测试发现的线索驱动深层测试，形成"浅层 → 深层"两轮推进模式
5. **结构化输出**：所有结果写成 JSON/Markdown 双格式，避免依赖终端输出（Windows 中文编码问题）

二、工具链最佳组合

2.1 推荐工具栈

角色	首选工具	说明
AI 主控	Claude Code (Opus 4.6, 1M context)	支持长上下文，可完整输入功能说明书；多模态能力可直接分析截图
浏览器自动化	Playwright (Python 或 TypeScript)	支持 networkidle、DOM 快照、全页截图、视频录制；对 Vue/React SPA 支持优于 Selenium
API 测试	Python requests / curl / Playwright page.evaluate()	requests 适合批量权限矩阵扫描；page.evaluate(fetch) 可直接复用浏览器 Token
实时探索	Playwright MCP	实时操控浏览器，AI 通过 browser_snapshot 获取 DOM 快照进行分析
静态分析	grep/正则 + JS Bundle 分析	从前端 JS Bundle 提取路由表和 API 端点
报告生成	Claude Code	将测试数据喂入 AI，自动生成规范化报告

2.2 工具组合策略

经多个参赛团队验证，**最有效的组合方式**是：

- **主力方案**：Playwright E2E 脚本（持久化、可回归、CI 友好）
- **辅助方案**：Playwright MCP 实时探索（灵活、适合发现未知问题）
- **安全专项**：Python requests 批量 API 测试（绕过前端限制，直接测后端）

> 注意：不要单独依赖 curl 测试 SPA 应用。SPA 的 HTML 只有空壳 `<div id="app"></div>` ，必须用真实浏览器渲染后才能获取有效内容。

三、Phase 1 -- 侦察（RECON）

目标：不登录也能知道"被测系统是什么"，建立测试的信息基础。

时间预算：15-30 分钟

3.1 页面侦察

```python
# Playwright 侦察脚本模板
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    # 1. 访问登录页，等待 JS 渲染完成
    page.goto("http://target:8180/login")
    page.wait_for_load_state("networkidle")

    # 2. 截取全页截图
    page.screenshot(path="recon/login_page.png", full_page=True)

    # 3. 提取所有表单元素
    inputs = page.eval_on_selector_all("input", """
        els => els.map(e => ({
            name: e.name, id: e.id, type: e.type,
            placeholder: e.placeholder
        }))
    """)

    # 4. 提取所有按钮
    buttons = page.eval_on_selector_all("button", """
        els => els.map(e => ({
            text: e.textContent.trim(),
            type: e.type, disabled: e.disabled
        }))
    """)

    # 5. 监听网络请求，发现 API 前缀
    api_calls = []
    page.on("response", lambda resp: api_calls.append({
        "url": resp.url, "status": resp.status
    }) if "/api" in resp.url else None)

    # 6. 识别技术栈
    tech_stack = page.evaluate("""() => ({
        vue: !!window.__VUE__,
        react: !!window.__REACT_DEVTOOLS_GLOBAL_HOOK__,
        arco: !!document.querySelector('[class*="arco-"]'),
```

```
        title: document.title
    })""")

    # 7. 将结果写入 JSON
    import json
    with open("recon/recon_report.json", "w", encoding="utf-8") as f:
        json.dump({"inputs": inputs, "buttons": buttons,
                   "api_calls": api_calls, "tech_stack": tech_stack},
                  f, ensure_ascii=False, indent=2)
```

3.2 JS Bundle 静态分析（关键技巧）

对于 SPA 应用，从前端 JS Bundle 中可以提取大量信息：

```
# 1. 获取 HTML，提取 JS 文件名
curl -s http://target:8180/ | grep -oE 'src="[^"]*\.js"'

# 2. 下载主 Bundle
curl -s http://target:8180/assets/index.xxx.js > /tmp/bundle.js

# 3. 提取路由配置
grep -oE 'path:"[^"]*"' /tmp/bundle.js | sort -u

# 4. 提取 API 端点
grep -oE 'url:"[^"]*"' /tmp/bundle.js | sort -u

# 5. 提取 API 前缀配置
grep -oE 'baseURL[=:]["'\'"][^"'\'']*' /tmp/bundle.js

# 6. 关键：扫描懒加载 Chunk（Vue 代码分割）
# 主 Bundle 可能只包含认证 API，业务 API 分散在各 Chunk 中
grep -oE 'import\("[^"]*"\)' /tmp/bundle.js  # 找到 chunk 文件名
# 逐一下载并提取 API 路径
```

> **最佳实践**：不要满足于主 Bundle 中发现的少量 API。Vue/React 的代码分割会把各模块 API 放在懒加载 chunk 中。扫描所有 chunk 文件才能获得完整 API 端点清单。（此技巧来自刘阔的 AI 自主探索发现）

3.3 登录与认证突破

```
# SPA 登录的常见障碍及解决方案

# 障碍1：弹窗遮挡登录按钮（如系统初始化向导 Modal）
page.evaluate("""
    document.querySelectorAll(
        '.arco-modal-wrapper, .arco-modal-mask, .arco-modal-container'
    ).forEach(el => el.remove())
""")

# 障碍2：Vue 响应式绑定不通过 input.value 触发
# 正确做法：使用 page.fill() 而非 JS 直接赋值
page.fill('input[type="text"]', username)
page.fill('input[type="password"]', password)

# 障碍3：按钮点击被拦截
# 使用 JS 直接触发 click 事件
page.evaluate("""
    const btns = document.querySelectorAll('button');
    for (let b of btns) {
        if (b.textContent.trim().includes('登录')) { b.click(); break; }
    }
""")

# 验证登录成功：通过 API 响应拦截比 URL 跳转更可靠
page.on("response", lambda resp:
    print("Login OK") if "getInfo" in resp.url and resp.status == 200 else None
)
```

3.4 侦察阶段产出

产出	说明
系统名称和技术栈	Vue/React + UI 框架 + 后端框架
API 前缀和端点清单	如 /prod-api/cosign/csDevice/list
路由地图	前端所有可达页面路径
认证机制	JWT Token 结构、有效期、算法
初始网络请求清单	系统启动时自动调用的接口

产出	说明
登录页截图 + HTML 快照	作为后续用例设计的基础

四、Phase 2 -- 建模（MODEL）

目标：将系统翻译成"可执行的测试清单"。

时间预算：30-60 分钟

4.1 功能说明书分析（文档驱动）

最佳 Prompt 模板：

```
你是一个资深软件测试工程师，精通 Web 应用测试、安全测试和自动化测试。

任务：阅读以下功能说明书，提取：
1. 所有业务模块（模块名、路径、优先级）
2. 每个模块的核心 API 接口（方法、路径、必填参数）
3. 系统中的状态机流转（状态值、转换条件、非法转换路径）
4. 权限节点清单（格式：模块:操作，如 cosign:csDevice:approve）
5. 文档中标注的"测试重点"或"潜在 Bug 线索"
6. 系统技术栈信息

输出格式：Markdown 表格，按模块分组

[将功能说明书全文粘贴于此]
```

> **关键经验**：将功能说明书完整放入 AI 的长上下文窗口（如 Opus 4.6 的 1M context），AI 在生成每个测试用例时都能参照原始需求，避免测试用例与实际需求脱节。

4.2 路由树建模

登录后通过 API 获取路由树，展平为可测试的功能清单：

```python
# 常见路由 API
# /prod-api/system/menu/getRouters
# /prod-api/user/menus
# /prod-api/auth/me

import json
routes_resp = requests.get(
    f"{BASE_URL}/prod-api/system/menu/getRouters?appId=cosign",
    headers={"Authorization": f"Bearer {token}"}
)
routes = routes_resp.json()

# 展平路由树为测试清单
def flatten_routes(menus, prefix=""):
    result = []
    for menu in menus:
        path = prefix + "/" + menu.get("path", "")
        result.append({
            "path": path,
            "component": menu.get("component", ""),
            "title": menu.get("meta", {}).get("title", ""),
            "permissions": menu.get("meta", {}).get("permissions", [])
        })
        if menu.get("children"):
            result.extend(flatten_routes(menu["children"], path))
    return result
```

4.3 JWT Token 分析

```python
import base64, json

# 解码 JWT Payload（Base64URL）
token_parts = token.split(".")
payload = json.loads(
    base64.urlsafe_b64decode(token_parts[1] + "==")
)

# 安全检查清单
checks = {
    "有 exp 字段": "exp" in payload,
    "有 iat 字段": "iat" in payload,
    "算法安全": token_header.get("alg") not in ["none", "HS256"],
    "不含敏感信息": "password" not in str(payload).lower(),
}
```

4.4 测试用例生成 Prompt 策略

按模块逐一生成（比一次性生成所有用例效果好 3 倍以上）：

```
基于以下模块信息，生成完整的功能测试用例。

模块：{模块名}
API 接口：{接口列表}
状态机：{状态流转描述}
权限节点：{权限列表}
已知风险点：{文档标注的注意事项}

要求：
- 正向场景覆盖主流程（至少 3 条）
- 异常场景占比 >= 40%（空值、超长、非法格式、边界值）
- 安全场景必含 XSS 注入和越权测试
- 状态机覆盖所有转换路径（含非法转换）
- 每条用例包含：ID、标题、前置条件、步骤、输入数据、预期结果、优先级

输出：Markdown 表格
```

4.5 用例设计原则

原则	说明	来源
等价类划分	每个输入字段至少 1 个有效类 + 2 个无效类	于小龙
边界值分析	最小值、最大值、最小值-1、最大值+1	于小龙
状态机覆盖	每条状态转换路径至少 1 条用例，含非法转换	于小龙/王绍瑞
权限矩阵	每个权限节点至少 1 个"有权"+ 1 个"无权"用例	卢雅/王立冬
负向场景优先	签名/密码管理系统的异常场景比正向更容易出 Bug	原丽俊

五、Phase 3 -- 执行（EXECUTE）

目标：将测试用例转化为可重跑的自动化脚本并执行。

时间预算：1-3 小时

5.1 脚本架构设计

推荐分层架构：

```
automation/scripts/
  ├── 01_recon.py              # 侦察脚本（独立可跑）
  ├── 02_login_test.py         # 登录模块测试
  ├── 03_module_explore.py     # 功能模块遍历 + 截图
  ├── 04_security_scan.py      # 安全专项测试
  ├── 05_permission_matrix.py  # 权限矩阵批量扫描
  └── config.py                # 全局配置（URL、账号、超时）
```

关键设计原则：

1. **单文件可独立运行**：每个脚本自包含，无隐式依赖
2. **结果结构化**：所有结果写成 JSON，不依赖终端打印
3. **幂等性**：只读或仅修改"已知被污染的字段"，不创造新脏数据
4. **失败不中断**：每个用例 try/except 隔离，单个失败只影响一条记录
5. **增量重跑**：拆分为多个小文件，上一步的 JSON 产物可作为下一步输入

5.2 功能模块标准化测试模板

```python
def test_module(page, path, module_name):
    """标准化功能模块测试模板（每个模块复用）"""
    results = []

    # 1. 导航
    page.goto(f"http://target:8180{path}")
    page.wait_for_load_state("networkidle")
    time.sleep(1.5)  # Vue 渲染等待

    # 2. 全页截图
    page.screenshot(
        path=f"screenshots/{module_name}.png",
        full_page=True
    )

    # 3. DOM 检测
    has_table = page.locator("table, .arco-table").count() > 0
    row_count = len(page.locator("tr").all())

    # 4. 操作入口验证
    buttons = [
        b.inner_text().strip()
        for b in page.locator("button").all()
        if b.is_visible() and b.inner_text().strip()
    ]

    # 5. 权限对比
    # 从 getInfo API 响应提取权限点，与 UI 按钮对应
    # 例：cosign:csDevice:remove -> 删除按钮应可见

    # 6. 记录结果
    results.append({
        "module": module_name,
        "path": path,
        "table_exists": has_table,
        "row_count": row_count,
        "buttons": buttons,
        "screenshot": f"screenshots/{module_name}.png"
    })
    return results
```

5.3 安全测试自动化

SQL 注入检测

```python
sql_payloads = [
    "' OR '1'='1",
    "1; DROP TABLE users;--",
    "' UNION SELECT NULL--",
    "1' AND SLEEP(3)--"  # 时间盲注
]

for payload in sql_payloads:
    t_start = time.time()
    resp = requests.post(f"{BASE}/auth/login", json={
        "username": payload, "password": payload
    })
    elapsed = time.time() - t_start

    # 判断标准
    if resp.json().get("code") == 200:
        report_bug("SQL 注入绕过登录", "CRITICAL")
    if elapsed > 3:
        report_bug("SQL 时间盲注", "CRITICAL")
    body = resp.text.lower()
    if any(kw in body for kw in ["sql", "exception", "mysql", "mybatis"]):
        report_bug("SQL 错误信息泄露", "HIGH")
```

XSS 检测（存储型 + 反射型）

```
xss_payloads = [
    '<script>alert(1)</script>',
    '<img src=x onerror=alert(1)>',
    '"><svg onload=alert(1)>'
]

# 存储型 XSS：写入后检查是否原样存储
for payload in xss_payloads:
    # 写入
    resp = requests.post(f"{BASE}/cosign/csDevice", json={
        "deviceName": payload, ...
    }, headers=auth_headers)

    # 读回
    list_resp = requests.get(f"{BASE}/cosign/csDevice/list",
                             headers=auth_headers)
    for row in list_resp.json().get("rows", []):
        if payload in row.get("deviceName", ""):
            report_bug(f"存储型 XSS: deviceName 字段未过滤", "HIGH")

# 反射型 XSS：检查搜索等参数是否被原样反射
page.fill("input.search", '<img src=x onerror=alert(1)>')
page.keyboard.press("Enter")
content = page.content()
if "onerror=alert" in content:
    report_bug("反射型 XSS", "HIGH")
```

权限矩阵批量扫描

```python
# 定义敏感接口列表
sensitive_apis = [
    {"method": "GET", "path": "/cosign/csHsmConfig/list"},
    {"method": "GET", "path": "/cosign/csAppKey/list"},
    {"method": "GET", "path": "/cosign/csDevice/list"},
    {"method": "GET", "path": "/system/user/list"},
    {"method": "GET", "path": "/system/role/list"},
]

# 三种 Token 场景
tokens = {
    "valid_admin": admin_token,
    "invalid_token": "Bearer invalid_fake_token_xxx",
    "no_token": None,
}

results = []
for api in sensitive_apis:
    for token_name, token_val in tokens.items():
        headers = {}
        if token_val:
            headers["Authorization"] = token_val
        resp = requests.get(f"{BASE}{api['path']}", headers=headers)
        is_secure = resp.status_code in [401, 403]
        results.append({
            "api": api["path"],
            "token": token_name,
            "status": resp.status_code,
            "secure": is_secure
        })
        if not is_secure and token_name != "valid_admin":
            report_bug(
                f"越权访问: {api['path']} 使用 {token_name} 返回 {resp.status_code}",
                "CRITICAL"
            )
```

敏感信息扫描

```
# 扫描所有 API 响应中的敏感字段
sensitive_patterns = [
    "password", "secret", "privateKey", "pin",
    "appSecret", "signKeyPassword", "activationCode"
]

for endpoint in all_endpoints:
    resp = requests.get(f"{BASE}{endpoint}", headers=auth_headers)
    resp_text = resp.text.lower()
    for pattern in sensitive_patterns:
        if pattern.lower() in resp_text:
            # 进一步检查是否为明文（非 null/masked）
            data = resp.json()
            if has_plaintext_value(data, pattern):
                report_bug(
                    f"敏感信息泄露: {endpoint} 返回 {pattern} 字段明文",
                    "CRITICAL"
                )
```

5.4 性能基准测试

```
# 基于 networkidle 的精确计时
def measure_load_time(page, path, module_name):
    t_start = time.time()
    page.goto(f"http://target:8180{path}")
    page.wait_for_load_state("networkidle")
    elapsed_ms = round((time.time() - t_start) * 1000)

    # 基准判断
    status = "PASS" if elapsed_ms < 2000 else \
             "WARN" if elapsed_ms < 5000 else "FAIL"
    return {"module": module_name, "load_ms": elapsed_ms, "status": status}
```

5.5 多视口响应式测试

```
viewports = [
    (375, 812, "mobile"),
    (768, 1024, "tablet"),
    (1440, 900, "desktop"),
    (1920, 1080, "wide"),
]

for width, height, name in viewports:
    page.set_viewport_size({"width": width, "height": height})
    page.goto("http://target:8180/dashboard")
    page.wait_for_load_state("networkidle")
    page.screenshot(path=f"screenshots/viewport_{name}.png", full_page=True)
```

六、Phase 4 -- 研判（TRIAGE）

目标：区分真 Bug 与误报，确保每个缺陷都可复现、可验证。

时间预算：30-60 分钟

6.1 研判原则

1. **每个 FAIL 都打开截图看一眼**：自动化断言可能误判（例如字符串 "404" 出现在 Base64 编码或 AppKey 中被误识别为 404 错误）
2. **每个 PASS 也要抽样**：侦察阶段的截图可能揭示 PASS 判定遗漏的问题（如偶发弹窗）
3. **误报与漏报同等重要**：AI 自动判定只是把人从重复劳动中解放，不替代人工判断
4. **区分测试脚本问题与系统缺陷**：测试失败时，先让 AI 分析是选择器过期、页面流程变更，还是真正的 Bug

6.2 AI 辅助失败分析 Prompt

```
以下测试执行失败：
[错误日志]
[失败截图]

请分析失败原因，判断属于以下哪种情况：
A. 测试脚本问题（选择器失效、等待时间不足等）→ 提供修复后的代码
B. 系统 Bug → 按以下格式记录：
   - 标题（一句话描述）
   - 严重程度（Critical/High/Medium/Low）
   - 复现步骤（含 curl 命令）
   - 预期结果
   - 实际结果
   - 影响分析
C. 环境问题（网络不通、代理干扰等）→ 说明排查方向
```

6.3 缺陷定级标准

等级	标准	示例
Critical	数据泄露、认证绕过、权限完全失效	AppSecret 明文拖库、无 Token 访问管理接口
High	核心安全缺陷、高危注入	暴力破解无防护、存储型 XSS、SQL 堆栈泄露
Medium	配置缺陷、中等安全风险	安全响应头缺失、验证码未开启、JWT 缺 exp
Low	非关键校验缺失、体验问题	错误码不规范、Nginx 版本泄露、非 HTTPS

6.4 Red-Green 迭代闭环

```
执行测试脚本
      │
      ▼
测试失败（Red）
      │
      ▼
AI 分析失败原因
      │
      ├── 选择器过期？ → 通过 MCP 获取新 DOM，更新选择器
      ├── 页面流程变更？ → 调整测试步骤
      └── 系统 Bug？ → 记录缺陷，调整断言为预期失败
      │
      ▼
重新执行（Green）
```

七、Phase 5 -- 沉淀（EVIDENCE）

目标：确保所有发现可追溯、可重跑、可被下一个人（或下一次 AI）10 分钟读懂。

7.1 截图命名规范

```
tc_<序号>_<场景>.png            # 测试用例截图
bug_<bugID>_<描述>.png          # 缺陷截图
module_<序号>_<模块名>.png      # 模块遍历截图
viewport_<设备类型>.png         # 响应式截图
```

7.2 缺陷报告格式

每个 Bug 必须包含四段式描述：

```
### BUG-<ID>: <标题>

**严重等级**: Critical / High / Medium / Low
**影响模块**: <模块名>

**复现步骤**:
1. 访问 http://target:8180/login
2. 使用 cosign_sec_admin / admin123 登录
3. 执行以下命令:
   ```bash
   curl -s -X GET "http://target:8180/prod-api/cosign/csAppKey/1" \
     -H "Authorization: Bearer $TOKEN"
```

4. 观察响应

预期结果：AppSecret 字段应脱敏显示（如 ****）
实际结果：AppSecret 字段返回完整明文密钥

影响分析：任意登录用户可遍历 ID 获取所有应用的签名密钥

修复建议：

- 后端 VO 层对 AppSecret 字段做脱敏处理
- 详情接口增加数据权限校验

```
### 7.3 交付物目录结构 {#73-交付物目录结构 }
```

```
test-project/
├── README.md # AI 工具版本 + 方法概述 + 成果自评
├── test-plan/
│ └── test-plan.md # 范围、策略、风险、进出准则
├── test-cases/
│ └── test-cases.md # 完整用例清单（含结果状态）
├── bug-report/
│ ├── bug-list.md # 缺陷清单（四段式描述）
│ └── screenshots/ # 所有截图证据
├── test-summary/
│ └── test-summary.md # 覆盖/质量/风险三段式总结
├── automation/
```
```

```
│ └── scripts/ # 可独立运行的自动化脚本
├── ai-evidence/
│ ├── session-logs/ # AI 会话日志（JSONL）
│ ├── prompts/ # 关键 Prompt 原文
│ └── config/ # AI 工具配置
└── methodology/
└── approach.md # 方法论文档
```

八、Prompt 工程最佳实践 {#八-prompt-工程最佳实践 }

8.1 高质量 Prompt 的三要素 {#81-高质量-prompt-的三要素 }

[角色定位] + [上下文信息] + [结构化输出要求]

低质量 Prompt：

帮我写登录测试用例

高质量 Prompt：

你是资深 Web 测试工程师，请为【协同签名系统登录模块】
生成测试用例，覆盖正向/边界/安全场景，
格式为 Markdown 表格（含 ID/标题/步骤/预期结果/优先级），
异常场景占比 >= 40%，至少包含 SQL 注入和 XSS 场景。

系统信息：

- 地址：http://192.168.6.184:8180
- 技术栈：Vue3 + Arco Design + RuoYi-Cloud
- 认证方式：JWT Token + 图形验证码
- 测试账号：cosign_sec_admin / admin123

8.2 Prompt 链策略 {#82-prompt-链策略 }

采用"Prompt 接力"，每步 AI 输出作为下步输入：

P-01 系统侦察结果 → 功能模块清单
↓
P-02 功能模块清单 + 说明书 → 测试计划
↓
P-03 测试计划 + 模块详情 → 测试用例（逐模块生成）
↓
P-04 测试用例 + DOM 快照 → Playwright 脚本
↓
P-05 执行结果 + 失败截图 → 失败分析与修复
↓
P-06 全量测试数据 → 规范化 Bug 报告
↓
P-07 Bug 报告 + 覆盖数据 → 测试总结报告

```
### 8.3 安全测试 Prompt 模板 {#83-安全测试-prompt-模板 }
```

针对以下系统，按 OWASP Top 10 优先级执行安全测试：

系统信息：

- 技术栈：Vue3 + Java RuoYi-Cloud
- 认证方式：JWT Token (HS512)
- 数据库：MySQL（推测）
- 业务类型：密钥管理系统（安全要求高）

按以下优先级测试（根据业务类型调整）：

1. 认证安全（密钥系统的第一道防线）
2. 敏感信息泄露（AppSecret、密码哈希、私钥）
3. 越权访问（IDOR、水平/垂直越权）
4. 存储型 XSS（设备名、CA DN、通知内容、拒绝原因）
5. SQL 注入（搜索框、过滤参数、orderByColumn）
6. 初始化接口重入（完成后是否还能调用）
7. 响应头安全配置检查

每个测试点输出：测试 payload、curl 命令、判断标准

8.4 Bug 发现高效 Prompt 模式 {#84-bug-发现高效-prompt-模式 }

模式	Prompt 示例	说明
业务规则验证	"检查设备在'已激活'状态下是否允许删除操作"	测试状态机边界
安全边界测试	"使用无效 Token / 其他角色 Token 访问接口"	越权检测
信息泄露扫描	"检查响应中是否包含 password/secret/key 等字段"	敏感数据
输入校验测试	"使用空值/超长/XSS/SQL 注入 payload 测试所有输入字段"	注入检测
数据一致性	"对比仪表盘统计数据与各模块列表 total 字段"	逻辑正确性
文档对比	"对比功能说明书描述与实际页面行为的差异"	需求偏差

九、AI 测试的已知边界与人工补充策略 {#九-ai-测试的已知边界与人工补充策略 }

9.1 AI 能做的 {#91-ai-能做的 }

- 快速生成和执行测试脚本
- 从 JS Bundle 提取路由和 API 地图
- 批量 API 安全扫描（越权、注入、泄露）
- 截图证据收集和视觉分析
- 测试报告自动生成和格式化
- 响应字段敏感信息自动检测
- 攻击链关联分析（多个 Bug 之间的关联）

9.2 AI 无法替代的 {#92-ai-无法替代的 }

场景	原因	补充策略
硬件交互（UKey、HSM）	需要物理设备	标记为 Out-of-Scope，仅测 UI 入口
视觉主观判断	UI 美观度、颜色协调性	人工审查截图
业务语义验证	签名结果是否密码学正确	结合专业密码学工具
XSS 实际触发确认	Vue 模板可能自动转义	需在真实浏览器中验证 v-html 场景
复杂多步骤跨页面工作流	AI 难以模拟完整用户行为	人工补充端到端测试
偶发性 Bug	触发条件不稳定	多次重复执行 + 条件组合
多角色协同场景	需要多账号同时操作	至少覆盖系统提供的所有角色

9.3 AI 容易遗漏的 UI 问题清单 {#93-ai-容易遗漏的-ui-问题清单 }

根据实际参赛发现，以下 UI 问题需要人工重点关注：

1. **偶发弹窗遮挡**：系统初始化向导 Modal 随机出现，遮挡正常操作
2. **数字动画残留**：仪表盘数字使用 CountUp 动画，截图时可能显示中间值

3. **中文编码问题**：Windows GBK 终端输出乱码（解决：`PYTHONIOENCODING=utf-8`）
4. **错别字/文案错误**：AI 视觉分析可发现，但需人工确认是否为真错
5. **字段空值/null 显示**：列表中某些字段显示为空或 "null" 字符串
6. **分页极端情况**：pageSize 无上限（可一次 dump 全表）
7. **表单联动失效**：A 字段选值后 B 字段选项未更新
8. **退出/注销功能**：下拉菜单可能被 Toast 通知遮挡

十、高价值 Bug 集中区（经验总结） {#十-高价值-bug-集中区经验总结 }

根据 15+ 个参赛团队的实际发现，以下是高价值 Bug 最容易出现的区域：

10.1 安全类 {#101-安全类 }

1. **操作日志接口**：最容易遗漏的敏感信息泄露点，可能记录密码明文
2. **初始化接口**：系统初始化完成后通常忘记限制匿名访问
3. **详情接口（GET /entity/{id}）**：列表接口做了脱敏，详情接口往往忘记
4. **批量操作接口**：pageSize 无上限可一次导出全表
5. **用户信息接口**：getInfo/getProfile 返回密码哈希
6. **权限控制**：前端隐藏按钮不等于后端拒绝（前端权限 != 后端权限）
7. **JWT Token**：缺少 exp 有效期、退出后 Token 仍可用

10.2 功能类 {#102-功能类 }

1. **状态机边界**：能否对"已激活"的设备执行删除？能否重复审批？
2. **数据一致性**：仪表盘统计数 vs 列表 total 不匹配
3. **删除级联**：删除 CA 后证书状态是否更新？
4. **默认值保护**：能否删除/停用默认 CA 或已激活 HSM？

10.3 测试深度对比 {#103-测试深度对比 }

深度层次	典型发现	推荐方法
第一层：界面可达性	404 页面、权限菜单不一致	Playwright 菜单遍历 + 截图
第二层：CRUD 正向	表单验证缺失、提交后无反馈	Playwright 表单操作
第三层：安全表面	暴力破解、XSS 注入、SQL 注入	curl / requests 批量测试
第四层：深度安全	IDOR 明文拖库、Mass Assignment	curl 逐接口逐字段验证
第五层：业务逻辑	状态机绕过、级联删除	API 状态组合测试

> **关键教训**（来自左俊祥）：不要用"列表接口已脱敏"直接推断"详情接口也已脱敏"。必须对每个实体的 list

十一、效率数据参考 {#十一-效率数据参考 }

多个参赛团队记录的实际效率数据：

阶段	AI 辅助耗时	传统人工估算	提速倍数
系统侦察 + 技术栈识别	15-30 分钟	1-2 小时	4-8x
测试用例设计（100+ 条）	1-2 小时	2-3 天	8-24x
自动化脚本编写	1-2 小时	1-2 天	8-16x
安全测试执行	30-60 分钟	1-2 天	16-48x
报告生成	30-60 分钟	4-8 小时	4-16x
总计	**4-8 小时**	**3-5 工作日**	**约 6-10x**

十二、创新技巧汇总 {#十二-创新技巧汇总 }

以下是从各参赛作品中提炼的创新做法：

技巧	来源	说明
JS Bundle 懒加载 Chunk 扫描	刘阔	不仅分析主 Bundle，还扫描所有懒加载 Chunk 提取隐藏 API
RMAE 四步循环	左俊祥	Recon-Model-Assert-Evidence 可复用方法论框架
Playwright MCP + E2E 双方案	李迪	探索式用 MCP，持久化用 E2E 脚本，二者互补
/create-test-project 向导	李迪	Claude Code Skill 自动初始化测试项目结构
截图回喂 AI 视觉分析	刘雪梅/高朋/王立冬	截图用 Read 工具喂回多模态 AI，发现 DOM 检查遗漏的 UI
权限矩阵批量自动化	卢雅/王立冬	82 个权限点 x 多角色 Token 批量扫描
文档 vs 实现对比	李迪	AI 将功能说明书与实际页面行为逐条对比
子代理驱动执行	姚磊	每个 Task 派发独立子代理，主代理协调调度
三层协作模型	高朋	Playwright codegen 录制 → AI 补全断言 → 人工补充
Prompt 链策略	原丽俊	P-01 到 P-12 逐步接力，避免一次性生成质量不高
已有脏数据检测	刘阔	不仅注入新 payload，还检查现有数据中的 XSS 残留
page.evaluate(fetch) API 测试	王绍瑞	通过浏览器 evaluate 执行 fetch，复用 localStorage Toke
攻击链分析	姚磊	识别多个 Bug 之间的关联（XSS + localStorage Token = 完整攻击链）

十三、快速复刻指南（30 分钟上手） {#十三-快速复刻指南30-分钟上手 }

环境准备 {#环境准备 }

```bash
```

```
# Python 方案
pip install playwright requests
python -m playwright install chromium
export PYTHONIOENCODING=utf-8  # Windows 必须

# 或 Node.js 方案
npm init -y
npm install @playwright/test
npx playwright install chromium
```

最小执行流程

```
# Step 1: 将本方法论和功能说明书一起输入 Claude Code
# Prompt: "进入 <URL>，账号 <USER> 密码 <PWD>，按照方法论五阶段执行测试"

# Step 2: AI 自动执行侦察 → 生成用例 → 编写脚本

# Step 3: 执行自动化测试
python automation/scripts/01_recon.py
python automation/scripts/02_login_test.py
python automation/scripts/03_module_explore.py
python automation/scripts/04_security_scan.py

# Step 4: 人工审查截图和结果 JSON

# Step 5: AI 生成规范化报告
# Prompt: "基于以下测试结果和截图，生成测试总结报告和 Bug 清单"
```

给 AI 的启动 Prompt

```
你是资深软件测试工程师。请对以下 Web 系统进行全面的功能 + 安全测试。

系统信息：
- URL：{系统地址}
- 账号：{用户名} / {密码}
- 测试范围：功能测试 + 安全测试 + UI 测试

执行方法论：RMAE 五阶段循环
1. 侦察：Playwright 访问系统，截图并分析 DOM/API/技术栈
2. 建模：分析路由树和权限，生成结构化测试用例
3. 执行：编写自动化脚本执行所有用例，结果写入 JSON
4. 研判：分析每个失败项，区分脚本问题和系统 Bug
5. 沉淀：整理截图证据、Bug 报告、测试总结

要求：
- 所有脚本单文件可独立运行
- 结果写入 JSON + Markdown 双格式
- 每个 Bug 都有 curl 复现命令
- 安全测试覆盖 OWASP Top 10
- 先执行浅层测试，发现线索后再深入
```

十四、常见陷阱与规避

陷阱	规避方法
直接让 AI 写 20 个用例而不先侦察	先让 AI 写一个 probe.py，拿到真实 DOM + API 结构后再写用例矩阵
信任 AI 的"想当然"路径	永远用 getRouters 的实际返回校正脚本中的路径
只测列表接口就认为"已脱敏"	对 list / {id} / POST / PUT / DELETE 五种入口都逐个验证
在 Windows 终端直接打印中文	设置 PYTHONIOENCODING=utf-8，结果写文件不写 stdout
断言用子串匹配（如 "404" in body）	改用正则 `re.search(r'\b404\b')` 或精确 selector
一个大文件包含所有测试	拆分为独立小文件，每个可单独重跑

陷阱	规避方法
测试数据污染生产环境	只读操作；写操作仅在已知被污染的字段上复用
忽略侦察阶段	侦察应占总时间 20-25%，避免后续大量返工
连续暴力破解触发账号锁定	错误密码测试仅执行 1-2 次，避免循环
Arco Design 动态类名不稳定	优先使用 text=、role= 选择器，避免依赖哈希类名

本方法论综合第五届 AI 编程实战大赛 15+ 个参赛作品的实战经验。
核心贡献者方法论来源：左俊祥(RMAE)、李迪(MCP+E2E)、刘阔(JS Bundle分析)、于小龙(文档驱动)、卢雅(权限矩阵)、王立冬(三层架构)、刘雪梅(Triage纪律)、原丽俊(Prompt链)。